

# ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №14

## Тема: Сортування Шелла

**Дисципліна:** Алгоритмізація і методи обчислень

**Спеціальність:** Професійна освіта (Цифрові технології)

**Тривалість:** 80 хвилин

## Мета роботи

Опанувати принцип роботи сортування Шелла як узагальнення сортування вставками з використанням послідовності кроків (gap sequence), навчитися застосовувати різні послідовності кроків (класичну та Кнута), дослідити на практиці залежність ефективності алгоритму від обраної послідовності.

## Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

## Короткі теоретичні відомості

**1. Принцип роботи.** Сортування Шелла (Shell sort), запропоноване Дональдом Шеллом у 1959 році, — це узагальнення сортування вставками. Головна ідея: замість того, щоб одразу сортувати весь масив вставками (порівнюючи лише сусідні елементи), спочатку виконується сортування вставками серед елементів, розташованих на певній відстані (кроці, gap) один від одного; крок поступово зменшується, і на останньому етапі крок дорівнює 1 — тобто виконується звичайне сортування вставками, але вже на **майже відсортованому** масиві (завдяки попереднім проходам з більшими кроками), що робить цей фінальний прохід значно швидшим за  $O(n^2)$ .

**2. Алгоритм для кроку gap:** масив розбивається на gap підпослідовностей (елементи з індексами  $i, i+gap, i+2\cdot gap, \dots$ ), і кожна підпослідовність сортується вставками незалежно (фактично — одним об'єднаним циклом з кроком gap замість кроку 1).

### 3. Послідовності кроків (gap sequences):

- **Класична послідовність Шелла:**  $n/2, n/4, n/8, \dots, 1$  (кожен наступний крок — половина попереднього). Проста в реалізації, але в найгіршому випадку дає складність  $O(n^2)$ .
- **Послідовність Кнута:**  $h = 3h+1$ , починаючи з  $h=1$  (тобто 1, 4, 13, 40, 121, ...), обрана у зворотному порядку до найбільшого  $h < n/3$ . Дає кращу практичну продуктивність, орієнтовна складність  $O(n^{1.5})$ .
- **Послідовність Хіббарда:**  $2^k - 1$  (1, 3, 7, 15, 31, ...), теоретична складність  $O(n^{1.5})$ .
- Загалом, точна асимптотична складність сортування Шелла **залежить від обраної послідовності кроків** і є предметом окремих досліджень — на відміну від більшості інших алгоритмів сортування, для сортування Шелла не існує єдиної "канонічної" оцінки складності.

**4. Чому сортування Шелла ефективніше за просте сортування вставками.** На перших етапах (з великим кроком) елементи, що знаходяться далеко від свого правильного місця, переміщуються одразу на велику відстань за один обмін (а не крок за кроком, як у звичайному сортуванні вставками), що значно швидше усуває "великі безлади" в масиві. До моменту, коли крок стає рівним 1, масив уже настільки близький до відсортованого, що фінальний прохід виконується практично за лінійний час.

**5. Властивості:** сортування Шелла — **нестійке** (елементи з однаковими ключами можуть змінити відносний порядок через переміщення на великі відстані) та **"на місці"** (in-place, не потребує суттєвої додаткової пам'яті).

## Хід роботи

### Завдання 1. Ручна трасировка сортування Шелла (класична послідовність)

Дано масив: [35, 12, 43, 8, 51, 6, 22, 90]. Виконати вручну сортування Шелла з класичною послідовністю кроків ( $n/2, n/4, \dots, 1$ ) для  $n=8$ , тобто кроки 4, 2, 1.

*Порядок виконання:*

1. Обчисліть послідовність кроків для  $n=8$ :  $8//2=4, 4//2=2, 2//2=1$ .
2. Для кроку  $gap=4$  виділіть підпослідовності елементів з індексами  $\{0,4\}, \{1,5\}, \{2,6\}, \{3,7\}$  і відсортуйте кожну підпослідовність (тут по 2 елементи) вставками — фактично просто порівняйте й, за потреби, поміняйте місцями кожну пару.
3. Запишіть стан усього масиву після завершення проходу з кроком 4.
4. Для кроку  $gap=2$  виділіть підпослідовності з індексами  $\{0,2,4,6\}$  та  $\{1,3,5,7\}$  (по 4 елементи кожна) і відсортуйте кожну вставками (порівнюючи елементи, віддалені на 2 позиції).
5. Запишіть стан масиву після проходу з кроком 2.
6. Для кроку  $gap=1$  виконайте звичайне сортування вставками (порівнюючи сусідні елементи) на вже частково впорядкованому масиві з попереднього кроку.
7. Запишіть остаточний відсортований масив і підрахуйте загальну кількість порівнянь за всі три проходи.

### Завдання 2. Побудова послідовності кроків Кнута та порівняльний розрахунок

Для масиву розміром  $n = 50$  обчислити послідовність кроків за формулою Кнута ( $h = 3h+1$ , починаючи з  $h=1$ , до найбільшого  $h < n/3$ ) та порівняти кількість кроків (проходів) із класичною послідовністю Шелла для того самого  $n$ .

*Порядок виконання:*

1. Обчисліть послідовність Кнута:  $h_0=1, h_1=3\cdot1+1=4, h_2=3\cdot4+1=13, h_3=3\cdot13+1=40$ . Оскільки  $40 < 50/3 \approx 16,67$  – неправильно; перевірте умову  $h < n/3$  і зупиніться на останньому  $h$ , що задовольняє умову (у цьому випадку  $h=13$ , оскільки  $40 > 16,67$ ).
2. Випишіть повну послідовність кроків Кнута у порядку використання (від найбільшого до 1): 13, 4, 1.
3. Обчисліть класичну послідовність Шелла для  $n=50$ : 25, 12, 6, 3, 1 (кожен наступний — цілочисельне ділення попереднього на 2).

4. Порівняйте кількість проходів (кроків) обох послідовностей: скільки проходів потребує класична послідовність і скільки — послідовність Кнута.
5. Зробіть висновок про те, чи завжди менша кількість проходів означає меншу загальну кількість операцій (підказка: кожен прохід із класичною послідовністю та послідовністю Кнута виконує різну кількість порівнянь всередині себе, тому пряме порівняння лише кількості проходів не дає повної картини — це буде перевірено експериментально в завданні 4).

### Завдання 3. Програмна реалізація сортування Шелла з класичною послідовністю

Реалізувати мовою Python сортування Шелла з класичною послідовністю кроків ( $n/2$ ,  $n/4$ , ..., 1), перевірити коректність.

*Порядок виконання:*

1. Напишіть функцію `shell_sort_classic(arr)`, що обчислює початковий крок `gap = n // 2`.
2. У зовнішньому циклі `while gap > 0`: для кожного `i` від `gap` до `n-1` виконайте вставку елемента `a[i]` у відсортовану частину з кроком `gap` (аналогічно звичайному сортуванню вставками, але порівнюючи `a[j]` з `a[j-gap]` замість `a[j-1]`).
3. Після завершення внутрішнього циклу зменшіть крок: `gap //= 2`.
4. Порахуйте кількість порівнянь і обмінів (зсувів) протягом усього процесу.
5. Перевірте функцію на тестовому масиві, порівнявши результат з `sorted()`.

*Приклад реалізації мовою Python:*

```
python
def shell_sort_classic(arr):
    a = arr.copy(); n = len(a); c = s = 0
    gap = n // 2
    while gap > 0:
        for i in range(gap, n):
            temp = a[i]
            j = i
            while j >= gap:
                c += 1
                if a[j-gap] > temp:
                    a[j] = a[j-gap]
                    j -= gap
                    s += 1
            else:
                break
            a[j] = temp
        gap //= 2
    return a, c, s

test = [8, 5, 2, 9, 1, 7, 3, 6, 4]
result, c, s = shell_sort_classic(test)
print("Результат:", result, " (порівнянь:", c, ", зсувів:", s, ")")
assert result == sorted(test)
print("Коректність підтверджено")
```

*Результат виконання програми:*

Результат: [1, 2, 3, 4, 5, 6, 7, 8, 9] (порівнянь: ..., зсувів: ...)  
Коректність підтверджено

#### Завдання 4. Порівняння послідовності Кнута з класичною та зі звичайним сортуванням вставками

Реалізувати сортування Шелла з послідовністю Кнута, порівняти кількість порівнянь трьох алгоритмів (звичайне сортування вставками, Shell з класичною послідовністю, Shell з послідовністю Кнута) для масивів зростаючого розміру.

*Порядок виконання:*

1. Напишіть функцію `shell_sort_knuth(arr)` за тією ж схемою, що й `shell_sort_classic`, але з попереднім обчисленням послідовності Кнута: почніть з `gap=1`, у циклі `while gap < n//3: gap = gap*3+1` знайдіть найбільший початковий крок; після кожного проходу зменшуйте `gap //= 3`.
2. Використайте раніше написану функцію `insertion_sort(arr)` (з практичної роботи №8) як базову лінію для порівняння.
3. Для розмірів масиву  $n \in \{500, 1000, 2000, 4000, 8000\}$  згенеруйте випадкові масиви й підрахуйте кількість порівнянь усіх трьох алгоритмів (звичайні вставки, Shell класичний, Shell Кнута).
4. Занесіть результати в таблицю "n — вставки(порівнянь) — Shell класичний(порівнянь) — Shell Кнута(порівнянь)".
5. Обчисліть, у скільки разів Shell-сортування ефективніше за звичайне сортування вставками для найбільшого дослідженого  $n$ , і порівняйте класичну послідовність із послідовністю Кнута.
6. Сформулюйте висновок про практичну доцільність використання Shell-сортування замість простого сортування вставками, а також про вплив вибору послідовності кроків на ефективність.

*Приклад реалізації мовою Python:*

```
python
def shell_sort_knuth(arr):
    a = arr.copy(); n = len(a); c = s = 0
    gap = 1
    while gap < n // 3:
        gap = gap * 3 + 1
    while gap >= 1:
        for i in range(gap, n):
            temp = a[i]
            j = i
            while j >= gap:
                c += 1
                if a[j-gap] > temp:
                    a[j] = a[j-gap]
                    j -= gap
                s += 1
            else:
                break
            a[j] = temp
        gap //= 3
    return a, c, s

import random
sizes = [500, 1000, 2000, 4000, 8000]
print(f"{'n':>6} {'вставки(порівн.):'>18} {'Shell класич.':>15} {'Shell Кнут':>12}")
for n in sizes:
    arr = [random.randint(0, 100000) for _ in range(n)]
    _, c_ins, _ = insertion_sort(arr)
    _, c_cl, _ = shell_sort_classic(arr)
    _, c_kn, _ = shell_sort_knuth(arr)
```

```
print(f"{n:>6} {c_ins:>18} {c_cl:>15} {c_kn:>12}")
```

*Результат виконання програми:*

| n    | вставки(порівн.) | Shell класич. | Shell Кнут |
|------|------------------|---------------|------------|
| 500  | 63952            | 6517          | 5916       |
| 1000 | 256765           | 15209         | 15148      |
| 2000 | 984729           | 35804         | 32364      |
| 4000 | 3984974          | 85144         | 75994      |
| 8000 | 16140532         | 205028        | 176239     |

*Висновок за результатами:* сортування Шелла значно ефективніше за звичайне сортування вставками – для  $n=8000$  воно виконує у  $\approx 79-92$  рази менше порівнянь (205028 і 176239 проти 16140532), незважаючи на те, що обидва алгоритми мають найгіршу асимптотичну складність  $O(n^2)$  у своїй базовій формі. Причина – сортування Шелла на попередніх (великокрокових) проходах усуває "далекі" безлади за набагато меншу кількість операцій, ніж це робив би прямий  $O(n^2)$  алгоритм. Послідовність Кнута також стабільно перевершує класичну послідовність ( $n//2$ ,  $n//4$ , ...) – приблизно на 10-14% менше порівнянь для великих  $n$ , що узгоджується з теоретичною оцінкою  $O(n^{1.5})$  для послідовності Кнута проти гіршої теоретичної оцінки  $O(n^2)$  для класичної послідовності в специфічних несприятливих випадках.

## Контрольні питання

1. Поясніть, чому сортування Шелла вважають узагальненням сортування вставками.
2. Що таке "крок" (gap) у сортуванні Шелла і як він змінюється протягом виконання алгоритму?
3. Чому фінальний прохід із кроком 1 виконується значно швидше, ніж якби це було звичайне сортування вставками з самого початку?
4. Назвіть відомі послідовності кроків для сортування Шелла та їхні орієнтовні асимптотичні оцінки складності.
5. Чому для сортування Шелла не існує єдиної "канонічної" асимптотичної оцінки складності, на відміну від багатьох інших алгоритмів?
6. Чи є сортування Шелла стійким алгоритмом? Обґрунтуйте.
7. Чи виконується сортування Шелла "на місці" (in-place)?

## Зміст звіту

1. Тема, мета роботи.
2. Розгорнута ручна трасировка (завдання 1) і розрахунок послідовностей кроків (завдання 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Порівняльна таблиця та висновки про ефективність різних послідовностей кроків.
5. Висновки.